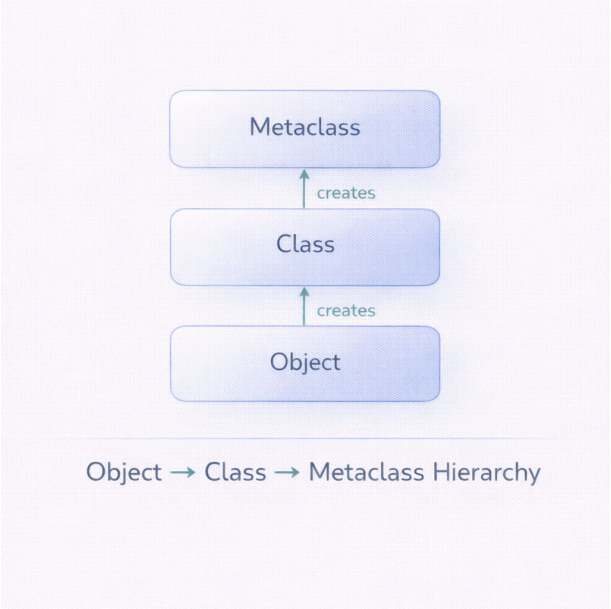
Python Programming Crash

Metaclasses

The Metaclass Model

Understand how Python creates classes and how metaclasses control that process.

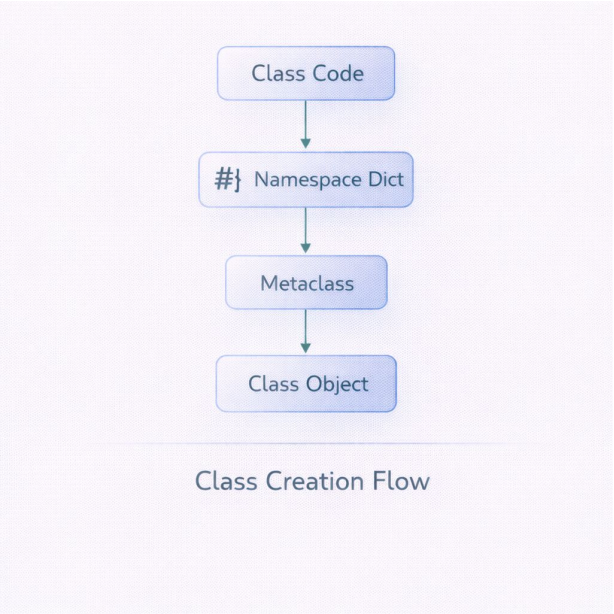
- Everything in Python is an object — including classes
- Classes are created using a **metaclass**
- Default metaclass is `type`
- Metaclasses define how classes behave
- Foundation for advanced frameworks and ORMs



Class Creation Flow

Python follows a structured process to create classes at runtime.

- Class definition is executed like normal code
- Python collects class attributes into a dictionary
- Metaclass is invoked to create the class
- `type()` is used by default
- Final class object is returned and assigned



Class Creation Flow (Example)

```
class User:
    name = "Wan"
```

Internally:

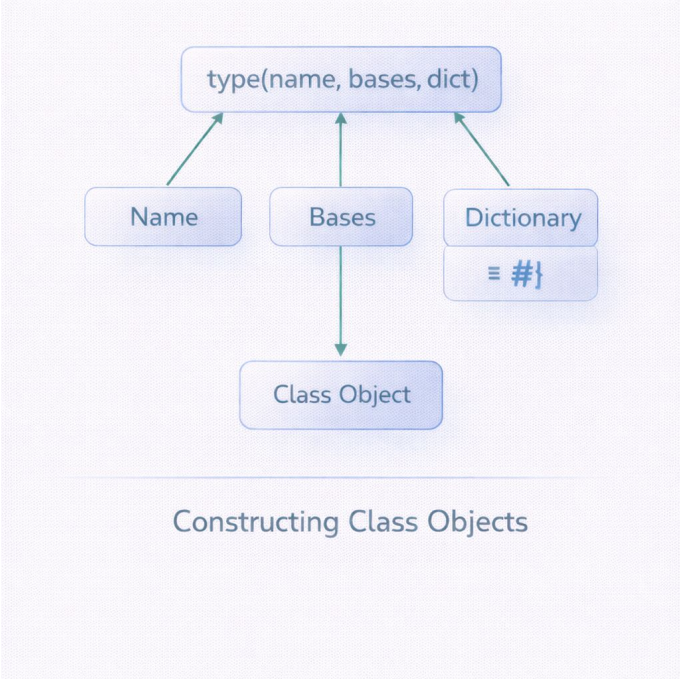
1. Create dict: `{"name": "Wan"}`
2. Call metaclass (`type`)
3. Return class object

→ Class creation is dynamic, not static

type() Internals

`type()` is the default metaclass that builds classes dynamically.

- `type()` can create classes programmatically
- Signature: `type(name, bases, dict)`
- Used internally for all class creation
- Enables dynamic class generation
- Foundation for understanding metaclasses



Demo: type() Internals

```
User = type("User", (), {"name": "Wan"})
```

Equivalent to:

```
class User:
    name = "Wan"
```

Emphasize:

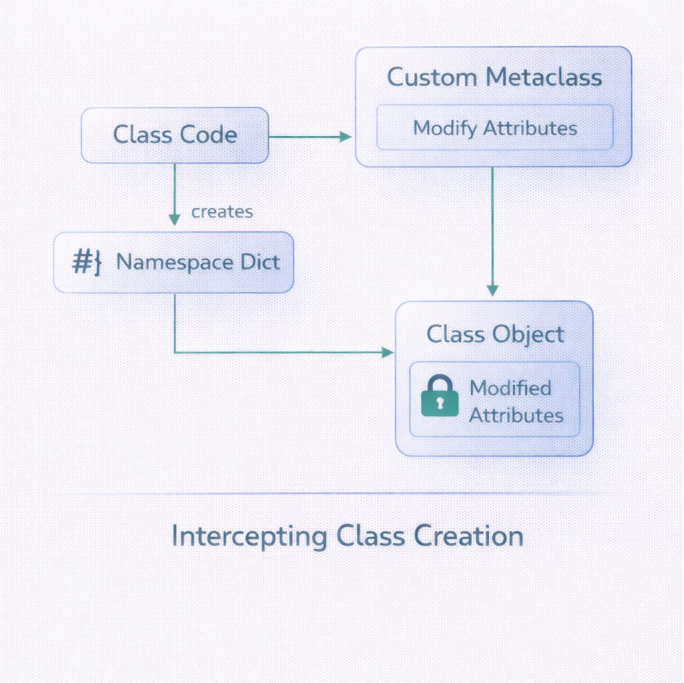
→ Classes are created at runtime

→ Not “compiled” like other languages

Custom Metaclasses

Define custom behavior during class creation using metaclasses.

- Metaclasses override class creation logic
- Implement using `class Meta(type)`
- Can modify class attributes automatically
- Enforce rules at class definition time
- Used in frameworks like Django ORM



Example: Custom Metaclasses

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        dct["created_by"] = "Meta"
        return super().__new__(cls, name, bases, dct)
```

Usage:

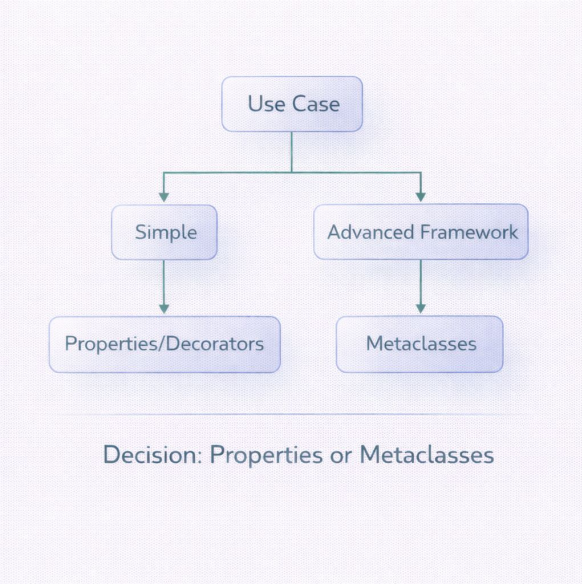
```
 class User(metaclass=Meta):
    pass
```

Emphasize:
→ Runs at class creation, not instance creation

When to Use Metaclasses

Use metaclasses only when you need to control class creation at a global level.

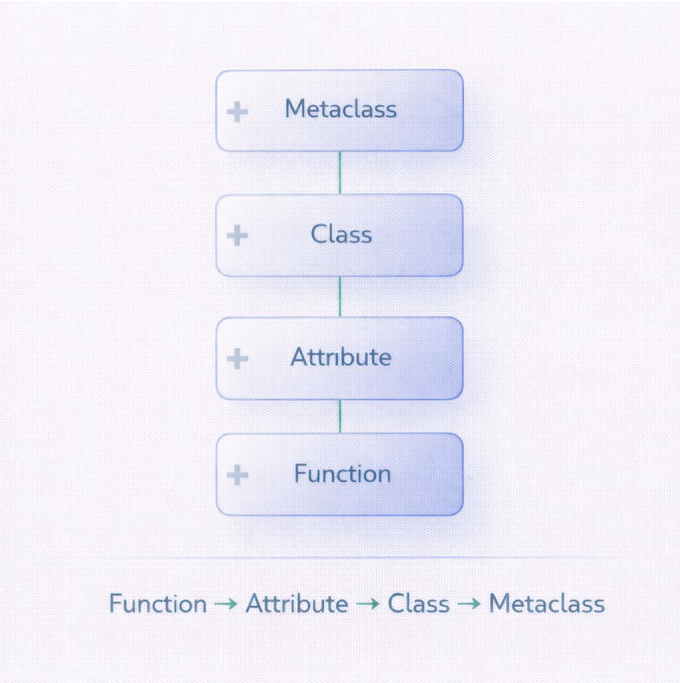
- Enforce rules across multiple classes
- Automatically modify or register classes
- Build frameworks and reusable systems
- Avoid for simple use cases (prefer decorators/properties)
- Use only when necessary due to complexity



Metaclasses vs Other Techniques

Choose the right tool: properties, decorators, descriptors, or metaclasses.

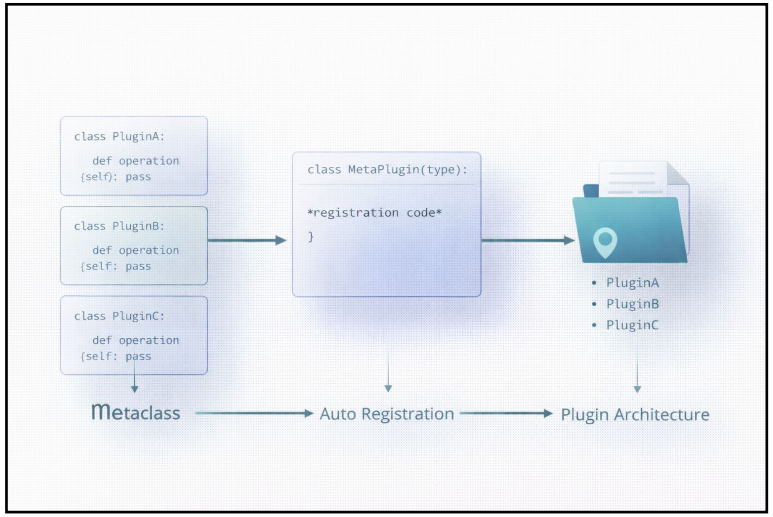
- Properties → control attribute access
- Decorators → modify function behavior
- Descriptors → reusable attribute logic
- Metaclasses → control class creation
- Each tool operates at a different level



Lab 8 — Auto-Register Classes via Metaclass

Build a custom metaclass that:

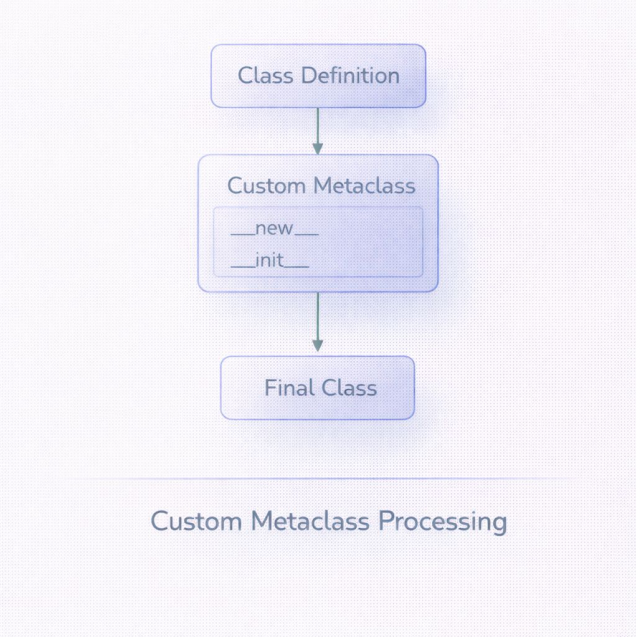
- automatically registers every class created with it
- stores registered classes in a global registry
- supports reusable plugin-style architecture



Creating Metaclasses

Define custom class creation behavior using `__new__` and `__init__` in metaclasses.

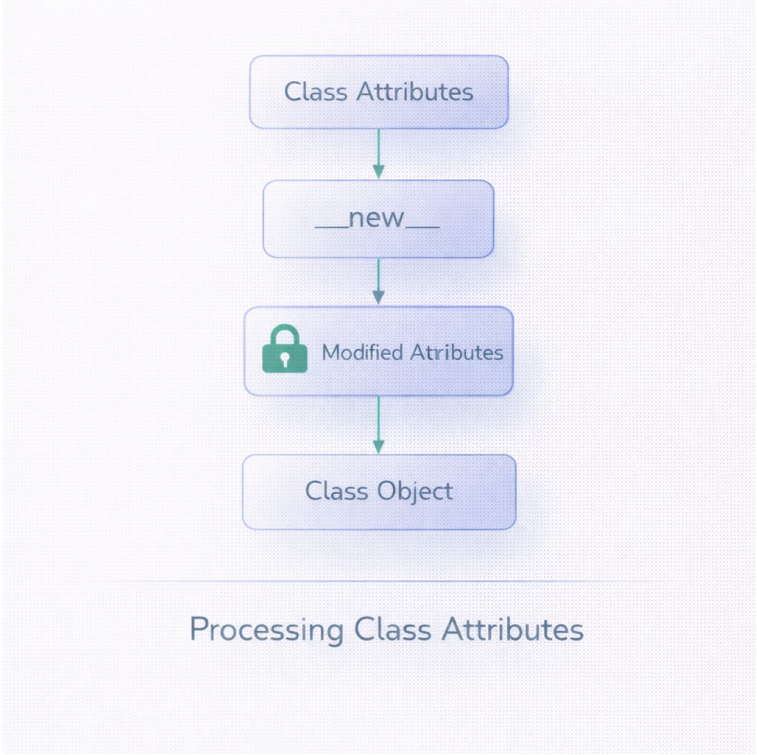
- Metaclasses inherit from `type`
- Override `__new__` to control class creation
- Override `__init__` to customize class initialization
- Intercept class definition before it is created
- Enable advanced control over class structure



new in Metaclasses

`__new__` is responsible for creating the class object.

- Called before the class is created
- Receives class name, bases, and attributes
- Can modify attributes before class creation
- Must return the new class object
- Most powerful hook in metaclasses



Example: **new** in Metaclasses

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        dct["created_by"] = "Meta"
        return super().__new__(cls, name, bases, dct)
```

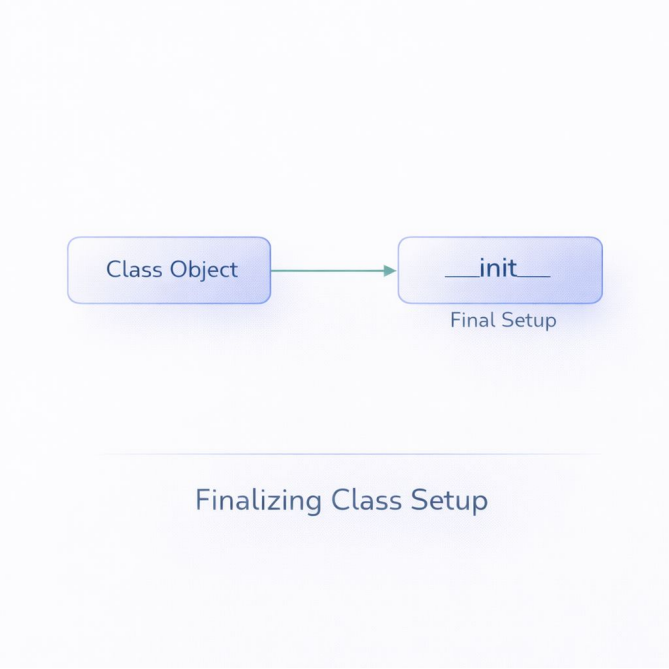
→ Modify class before it exists

- Key insight:
  → This is where most logic happens

init in Metaclasses

`__init__` configures the class after it has been created.

- Called after `__new__`
- Receives the created class object
- Used for post-processing and setup
- Less commonly used than `__new__`
- Can enforce rules or log class creation



Example: **init** in Metaclasses

```
class Meta(type):
    def __init__(cls, name, bases, dct):
        print(f"Class {name} created")
        super().__init__(name, bases, dct)
```

Emphasize:

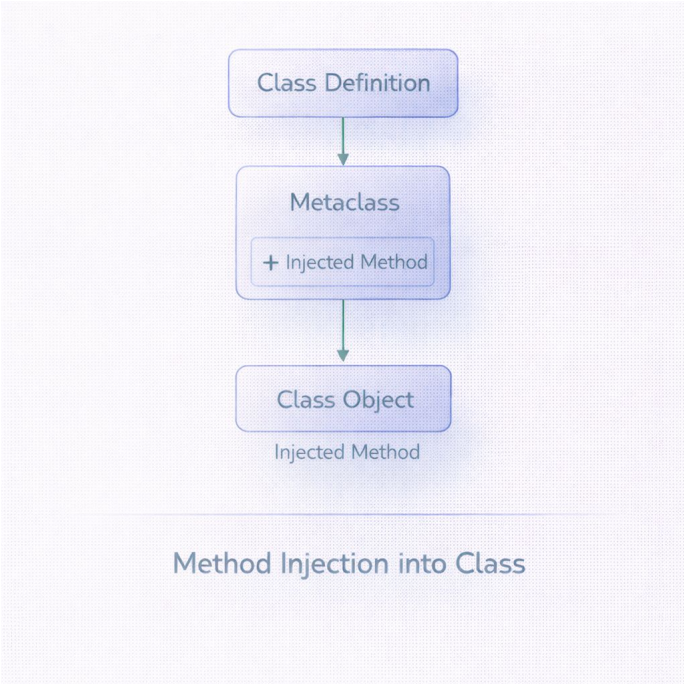
→ `__new__` creates

→ `__init__` configures

Injecting Methods

Automatically add methods or attributes to classes during creation.

- Modify class dictionary inside `__new__`
- Add reusable methods across multiple classes
- Enforce consistent behavior
- Reduce repetitive code
- Useful in frameworks and base models



Example:Injecting Methods

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        def greet(self):
            return "Hello from injected method"
        dct["greet"] = greet
        return super().__new__(cls, name, bases, dct)
```

Usage:

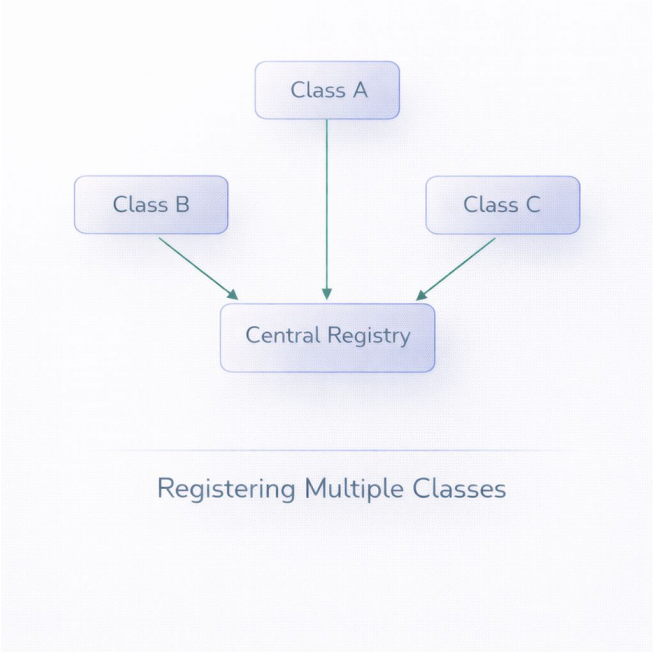
→ All classes using this metaclass get `greet()` automatically

→ DRY principle at class level

Automatic Registration

Automatically track or register classes when they are defined.

- Maintain a registry of all classes
- Useful for plugins, models, or handlers
- Happens at class creation time
- Enables dynamic discovery of components
- Common in frameworks and large systems



Example Automatic Registration

```
registry = []

class Meta(type):
    def __new__(cls, name, bases, dct):
        new_class = super().__new__(cls, name, bases, dct)
        registry.append(new_class)
        return new_class
```

→ Every class is automatically tracked

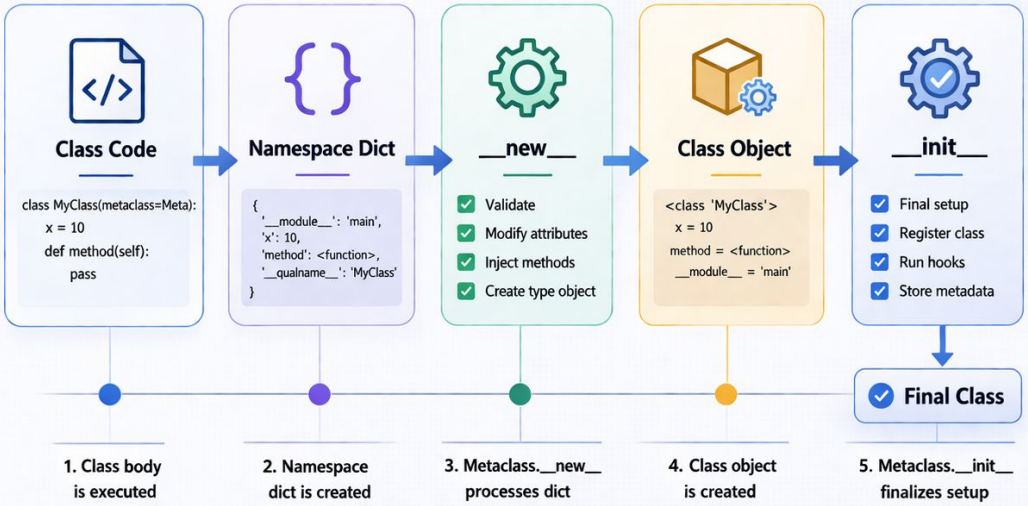
- Real-world:
  → Django models, plugin systems

Metaclass Creation Flow (Summary)

Understand the full lifecycle of class creation using metaclasses.

- Class definition is executed
- Attributes collected into dictionary
- `__new__` modifies and creates class
- `__init__` configures class
- Final class is returned and usable

End-to-End Class Creation Flow

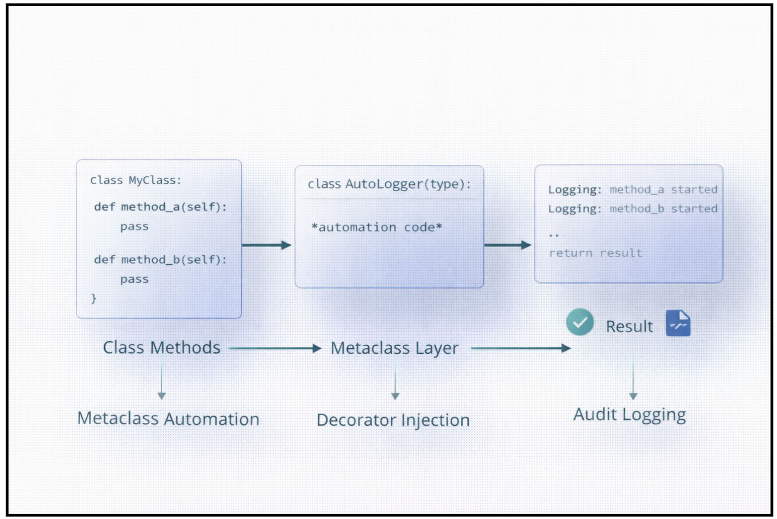


What Happens

Lab 9 - Auto-Inject Logging Methods

Build a custom metaclass that:

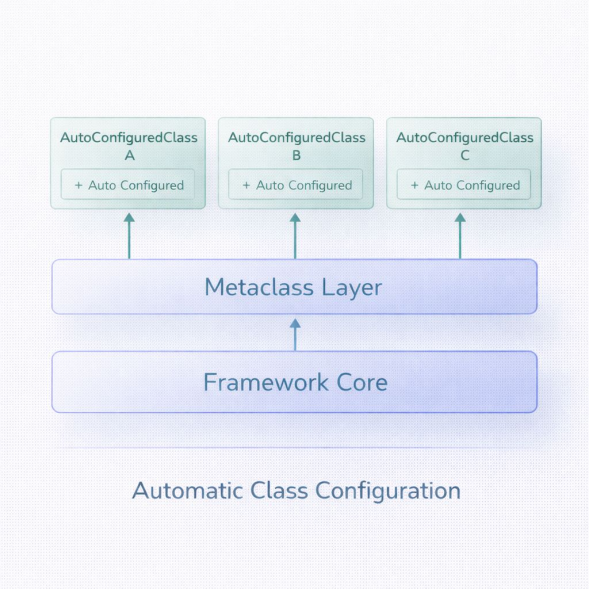
- automatically wraps class methods with a logging decorator
- adds audit-style logging without modifying each method manually
- demonstrates class-level automation using metaclasses



Metaclasses in Real-World Systems

Metaclasses power advanced frameworks by controlling how classes are defined and registered.

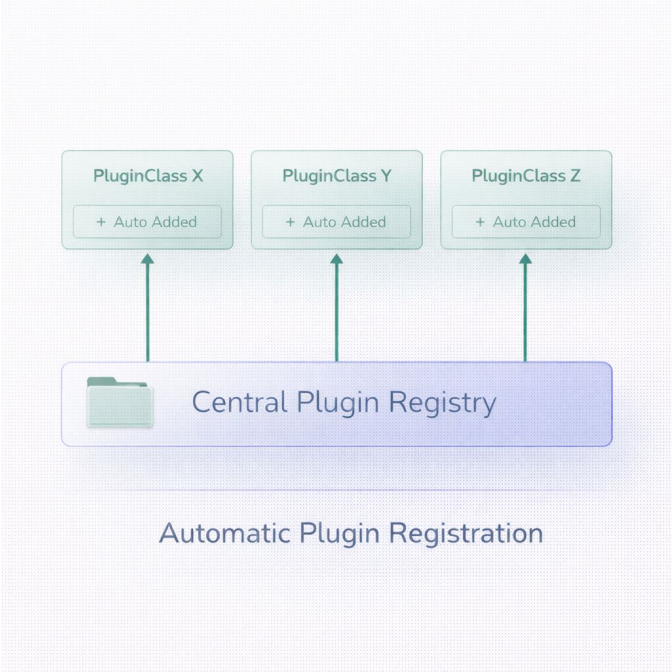
- Used behind the scenes in many Python frameworks
- Enable automation at the class level
- Reduce boilerplate code in large systems
- Support dynamic discovery and configuration
- Essential for scalable and extensible architectures



Plugin Systems

Automatically register and manage plugins using metaclasses.

- Classes are auto-registered when defined
- Enables dynamic plugin discovery
- No manual registration required
- Supports modular and extensible design
- Common in CLI tools, AI agents, and extensions



Example concept: Plugin Systems

```
plugins = []

class PluginMeta(type):
    def __new__(cls, name, bases, dct):
        new_class = super().__new__(cls, name, bases, dct)
        plugins.append(new_class)
        return new_class
```

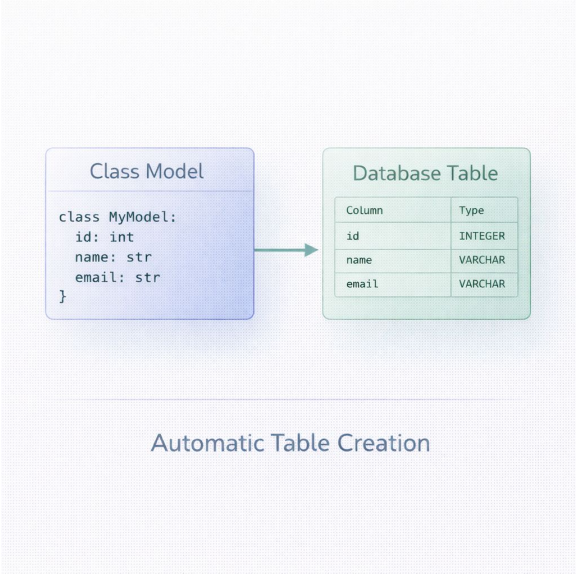
→ “Drop a class → automatically available”

- Real-world:
  → plugin-based systems, AI agents, extensions

ORM Systems

Map class definitions to database tables automatically using metaclasses.

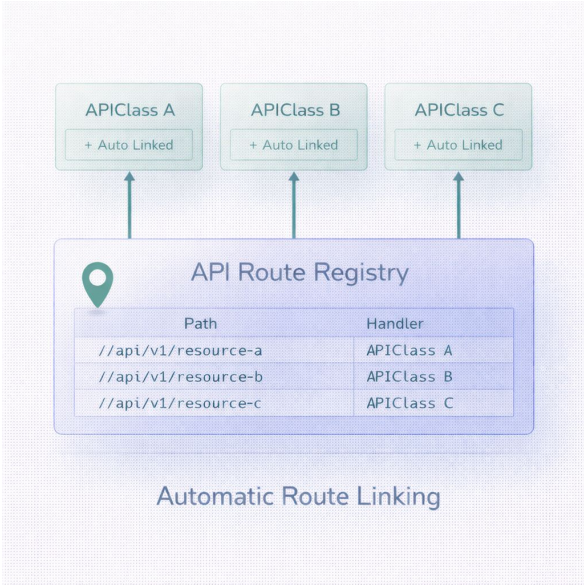
- Convert class attributes into database fields
- Automatically register models
- Enforce schema rules at class creation
- Enable query building and validation
- Core mechanism in ORMs like Django



API Auto-Registration

Automatically register API endpoints or handlers using metaclasses.

- Detect classes and register routes automatically
- Reduce manual configuration
- Enable scalable API design
- Useful in microservices and frameworks
- Improves maintainability and consistency



Example : API Auto-Registration

Concept:
→ Define class → route auto-created

- Example idea:

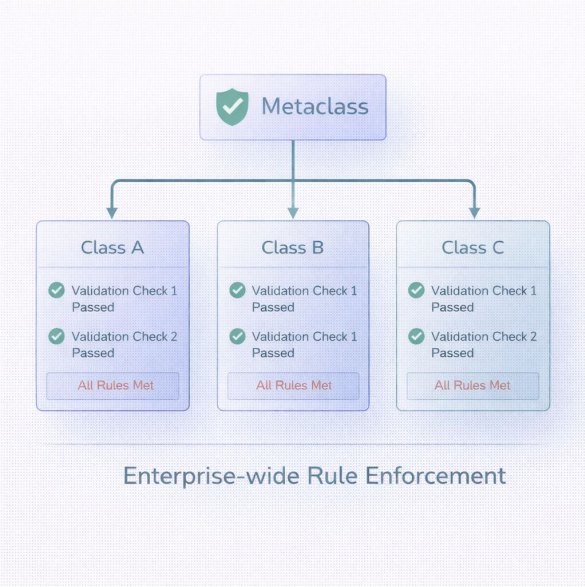
```
routes = []

class APIMeta(type):
    def __new__(cls, name, bases, dct):
        new_class = super().__new__(cls, name, bases, dct)
        routes.append(new_class)
        return new_class
```

- Real-world:
  → internal frameworks, automation tools

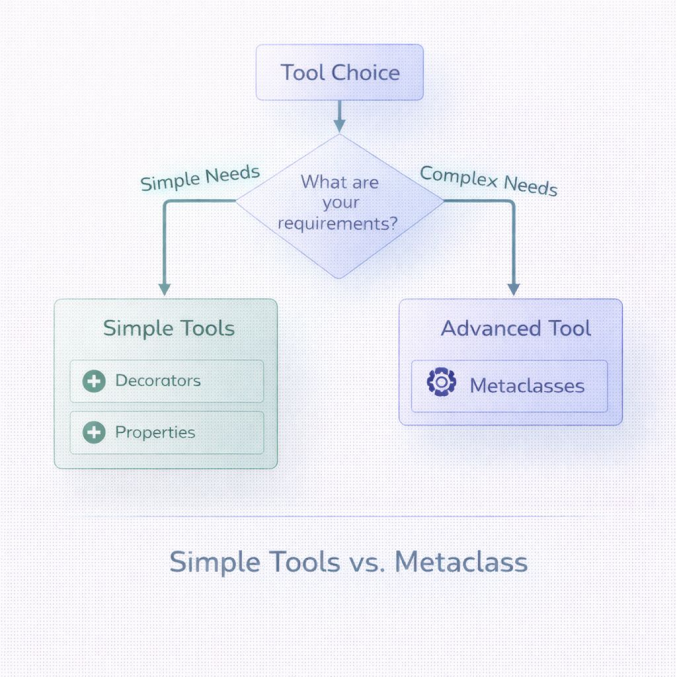
Framework-Level Control

Use metaclasses to enforce rules and structure across an entire system.

- Enforce naming conventions or required attributes
- Validate class definitions at creation time
- Apply global rules across all classes
- Centralize system behavior
- Used in large-scale enterprise frameworks



When Metaclasses Shine

Use metaclasses for automation, consistency, and system-wide control.

- Best for frameworks and reusable systems
- Ideal for automatic registration and validation
- Not needed for simple applications
- Adds power but increases complexity
- Use when other techniques are insufficient